Development and Evaluation of Application Scenarios for a Quadruped Manipulator in NVIDIA Isaac Sim

Firdavs Shukurilloev

***Abstract*— Quadruped robots with an arm can drive to a shelf, lower their body and pick up an object, but simulation work usually tests walking and grasping one at a time. In this project we built a complete simulation setup for the lab's Unitree B2W wheeled quadruped with a Z1 arm and gripper in NVIDIA Isaac Sim. It contains a digital twin of the robot with the sensors we plan to mount on the real one (two Intel RealSense cameras and an Ouster lidar), a pipeline that turns phone photos of real rooms into photoreal scenes, whole-body controllers trained with reinforcement learning, and warehouse fetch scenarios. In these scenarios the robot follows an instruction such as *"Pick up the red box on the green table and bring it to me"*: a vision-and-language navigation model walks it towards the table, Nav2 parks it next to the box, the arm picks the box up and, when asked, the robot carries it back. In more than 800 simulated episodes the robot never fell, and in the final 30 recorded episodes it picked the requested box every time. Each episode is saved with camera images, language, robot state and whole-body actions in the LeRobot format, as a first dataset for training a whole-body vision-language-action model. The code is available at `https://github.com/sheff-hub/b2w-z1-isaacsim`.**

***Index Terms*— Legged robots, mobile manipulation, simulation, reinforcement learning, vision-and-language navigation.**

I. Introduction

Wheeled quadrupeds with an arm combine fast and efficient driving with legs that can lower and tilt the body. This makes them a good fit for fetching objects in warehouses, shops and homes [1]. Their value lies in how the parts work together: the robot drives to a table, adjusts its body so that the arm can reach, picks up an object and carries it away. GPU-accelerated simulators such as Isaac Sim and Isaac Lab [2], [3] make it cheap to build and test such behaviour before it runs on hardware.

Most work so far studies one skill at a time: a walking controller, a reaching policy or a navigation model, each with its own metric. Complete application scenarios, in which the robot receives a task in plain language and has to navigate, pick and deliver, are rare. So are datasets that record such scenarios in a form that a learned whole-body controller can use. This report describes a Forschungspraktikum that worked on both points for the lab's Unitree B2W with a Z1 arm. Fig. 1 shows the outcome: from one sentence, the robot walks to the named table, parks next to the box, picks it up and brings it back.

A. Related Work

Simulation. GPU-parallel simulators [3], [4] made legged locomotion trainable in minutes [5]. Embodied-AI benchmarks such as Habitat 2.0 [6], BEHAVIOR-1K [7] and ManiSkill3 [8] define household and rearrangement tasks, but mostly for wheeled or fixed-base robots with simplified locomotion. Our scenarios keep the legged-wheeled dynamics and the robot's own sensors in the loop.

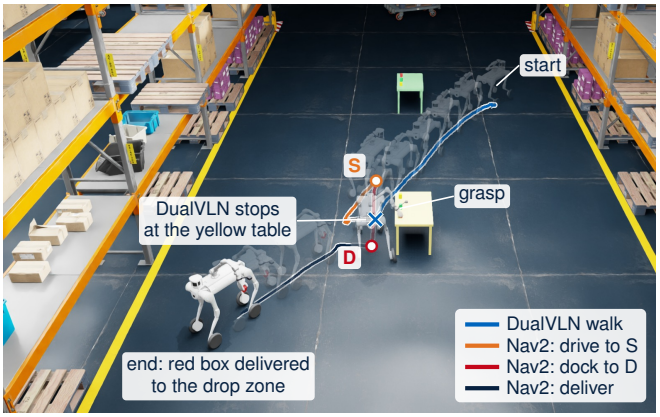


Fig. 1. Our Unitree B2W with Z1 arm fetching a box in NVIDIA Isaac Sim, for the task *"Pick up the red box on the yellow table and bring it to me"*. DualVLN walks it from the far end of the aisle to the yellow table. Nav2 then drives it to the staging pose S beside the table, docks it at the pick point D and, after the pick, carries the box to the drop zone.

Whole-body loco-manipulation. Model-based controllers coordinate base and arm with explicit constraints [9]–[11]. Learning-based methods train one policy for legs and arm [12], [13] or combine a learned walking policy with a model-based arm [14]. Jiang et al. [15] track a full 6D hand pose on a wheeled quadruped with a Reward Fusion Module (RFM), which we re-implemented for our robot. Navigation for wheeled-legged robots is studied in [16], [17].

Language, navigation and data. Vision-and-language navigation moved from graphs [18] to continuous worlds [19]; DualVLN [20] pairs a slow vision-language model (VLM) that picks a goal with a fast policy that draws the path. VLA models [21]–[23] learn from large demonstration sets [24], often stored in the LeRobot format [25]. Gao et al. [1] propose to collect whole-body data for quadruped manipulators in two stages: first with scripted experts, then with pretrained navigation and VLA models driving the robot. Our pipeline is a first implementation of this idea in simulation. TargetBench [26] evaluates planning towards semantic targets on quadruped data.

Forschungspraktikum report, TUM AVS Lab, October 2026. Supervisors: Yuan Gao and Prof. Dr.-Ing. Johannes Betz.

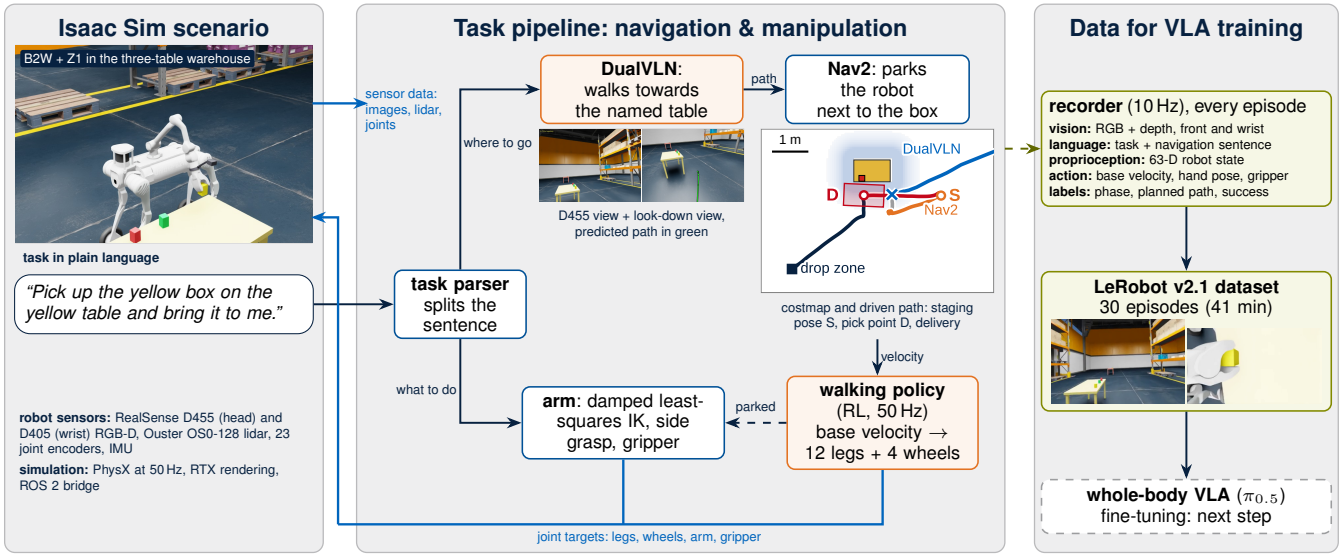


Fig. 2. System overview. A task in plain language is split into a navigation sentence and a task type. DualVLN walks the robot towards the named table, Nav2 parks it next to the box, and the arm picks the box with inverse kinematics; a learned walking policy turns all base velocity commands into leg and wheel targets. The simulated robot closes the loop with its camera images, lidar scans and joint states. Every signal is recorded and stored as a LeRobot dataset for training a whole-body VLA model. The DualVLN view and the Nav2 sketch come from the episode of Fig. 1: DualVLN heads for the yellow table and stops beside it (×); the robot backs away from the table, and Nav2 drives it to S, docks it along the table edge at D, where it stands beside the red box (red outline), and later delivers the box.

TABLE I

EXPECTED DELIVERABLES OF THE PROJECT AND OUR CONTRIBUTIONS.

Deliverable	Contribution
Integrated robot setup (USD, sensors, control)	B2W + Z1 + gripper with 23 degrees of freedom (DoF), D455, D405 on a printed holder, OS0 lidar, ROS 2; walking and whole-body policies (Section II-A, Section II-C)
Application scenarios in Isaac Sim	warehouse scenarios S1–S4 with language tasks; photoreal home and lounge from phone photos (Section II-D, Section II-B)
Navigation and manipulation pipelines	scripted, Nav2 and DualVLN navigation; side pick and delivery (Section II-E)
Evaluation across scenarios and tasks	more than 800 seeded episodes, controller tests, recorded datasets (Section III)
Documented code base	Isaac Lab tasks and training, fetch pipeline, Nav2 and DualVLN bridges, recorder and LeRobot tools, robot build scripts, one-command reconstruction, with READMEs

B. Gaps and Contributions

At the start of the project there was no simulated B2W with the Z1 arm that runs complete tasks with realistic sensors, no scenario that combines navigation and manipulation with a language task, and no dataset of such episodes. Our contributions follow the deliverables of the project (Table I): an integrated robot setup in Isaac Sim, application scenarios with photoreal rooms, navigation and manipulation pipelines driven by language, and an evaluation over more than 800 episodes together with a recorder that stores every episode for VLA training.

II. Methodology

Our system has four parts (Fig. 2): a digital twin of the robot with its sensors, photoreal scenes of real rooms, whole-body controllers, and application scenarios with task pipelines that turn a sentence into a fetched box and record every episode as training data.

A. Digital Twin of the Robot

We converted Unitree's URDF files of the B2W and the Z1 to USD and joined them into a single articulation, so that legs, wheels and arm are simulated as one system. This is a precondition for whole-body control. The import caused several faults that gave no error message. The gripper meshes pointed to `package://` paths the importer could not find, so the gripper was invisible. The importer wrote drives that let the wrist sag by about 50° under the gripper's weight, and the stowed gripper sat inside the lidar cage. A build script now regenerates the robot file from the original model and fixes all of this: a local URDF with proper jaw colliders, stiff joint drives, the arm moved 11 cm back, and two collision pairs that overlap by design excluded from self-collision. The final robot has 23 DoF: 12 leg joints, 4 driven wheels, 6 arm joints and the gripper. The same steps produced a second variant, the legged B2 with the Z1 arm. Action graphs publish the camera images, the lidar point cloud, the joint states and the transform tree to ROS 2, which we checked in RViz2 on the host.

The sensors match the hardware we plan to mount on the real robot (Fig. 3). The head carries a RealSense D455 camera, for which we use the lens of Isaac Sim's own D455 model (90° wide, 16:10). The wrist carries a RealSense D405, a short-range camera meant for grippers, modelled

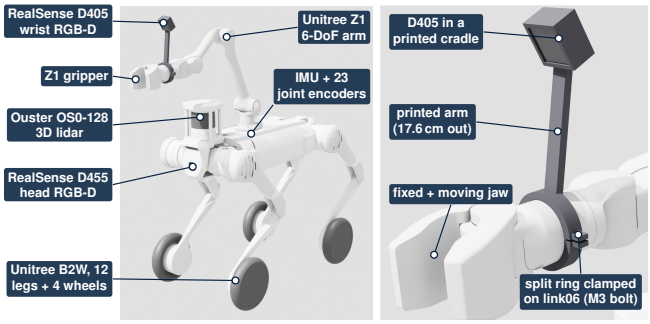


Fig. 3. Our simulated robot, Unitree B2W with Z1 arm and gripper, and its sensors, which are models of the hardware planned for the real robot. Left: the label lines end at the simulated sensor positions. Right: the wrist camera holder we designed, a split ring clamped around the last arm link with a printed arm and a cradle for the RealSense D405. From there the camera sees the whole gripper.

with Intel's mesh [27], its real field of view of 87° and its minimum depth of 7 cm. We designed a 3D-printable holder for it: a split ring clamped around the last arm link, an arm out to the side and a cradle around the camera. The camera position was chosen by hand so that the whole gripper is in view. We checked that the holder stays clear of the jaws over their full opening range, of the arm over a full turn of the wrist, of the body when the arm is stowed, and of the table and boxes by replaying 55 recorded picks with the holder attached. An Ouster OS0-128 lidar feeds Nav2, and joint encoders and an IMU give the robot state. All sensors publish to ROS 2.

B. Photoreal Scenes from Phone Photos

To get home-like scenes, we reconstructed two real rooms, a home and a lounge, from phone photos with NVIDIA's NuRec workflow [28] (Fig. 4). COLMAP [29] estimates the camera poses and a sparse point cloud, 3DGUT [30] with MCMC densification [31] trains a Gaussian splatting model [32]. We load the exported assets in Isaac Sim 6.0.1 rather than 5.1.0, because it renders them better. The stock export tilts the scene by 90° and writes positions that overflow 16-bit floats; our export fixes both. One script runs the whole chain unattended: it checks the photos, runs COLMAP, sets the number of Gaussians from the number of COLMAP points, trains, exports and checks the asset. A short file per room holds its settings, and an interrupted run continues where it stopped.

C. Whole-Body Control

We trained controllers for the 22 joints of the B2W with the Z1 arm with proximal policy optimization (PPO) [33] in Isaac Lab. A policy runs at 50 Hz. It sees how the body turns and tilts, the joint states, its last action and a command, and it outputs position targets for legs and arm and velocity targets for the wheels. We changed one thing per training run, about 30 runs in total, and watched every result in simulation instead of trusting the reward curve alone. The design went through three steps:

1) *One policy instead of two:* a single policy drives and reaches at the same time, instead of a walking policy and a reaching policy running side by side.
2) *Targets fixed in the world:* the hand target stays fixed in the world instead of moving with the robot, so the policy has to use its body to reach it.
3) *One 6D command:* following Jiang et al. [15], the policy receives only a 6D hand pose, up to 3 m away in any direction, and no velocity command. It decides by itself whether to turn, drive, crouch or stretch the arm.

For the last step we re-implemented the Reward Fusion Module (RFM), which shifts the reward from driving to manipulation as the hand gets closer, and replaced its distance-based switch with a sphere around the shoulder that has the size of the arm's reach. The final policy commands small changes of the arm targets instead of the targets themselves (Section III-C). These policies are not part of the fetch pipeline yet; for the pick we use a scripted arm (Section II-E).

D. Application Scenarios

All scenarios use the Isaac Sim warehouse with tables of 0.6 m height (Fig. 5):

1) **S1, single table:** one box at a random pose on one table, and a drop zone across the aisle.
2) **S2, three tables:** a red, a yellow and a green table, each with a red, a yellow and a green box in a random order. The task names a box and a table.
3) **S3, language:** either scene with an instruction such as *"Pick up the red box on the green table and bring it to me"*. Half of the tasks ask only for the pick, the other half also for the delivery, and the sentence says which.
4) **S4, live:** a GUI in which a user types tasks such as *"go to the green table and pick the red box"* or *"bring it back"* and watches the robot do them.

Box poses are always random, and in most runs the start pose, the target and the wording are random too.

E. Task Pipelines

The robot first has to get to the box, then pick it up and, when asked, carry it back (Fig. 2).

1) Walking controller: for driving we use ETH's walking policy for the real B2W from the SRU project [34], an LSTM network that turns velocity commands into leg and wheel targets at 50 Hz. It pulled the rear legs inwards after turns, so we fine-tuned it with reinforcement learning (RL), rewarding it for staying close to the original policy and for keeping a wide stance while driving straight.

2) Navigation: we compare three ways of getting to the box. *Scripted:* a hand-written state machine drives via waypoints around the tables and corrects the final position in closed loop. It is precise but tuned to this scene. *Nav2:* the ROS 2 navigation stack [35] plans a path to a staging pose S in front of the box, follows it with Regulated Pure Pursuit [36] and then uses its docking controller to drive the last 2 m to the pick point D, with the table on the

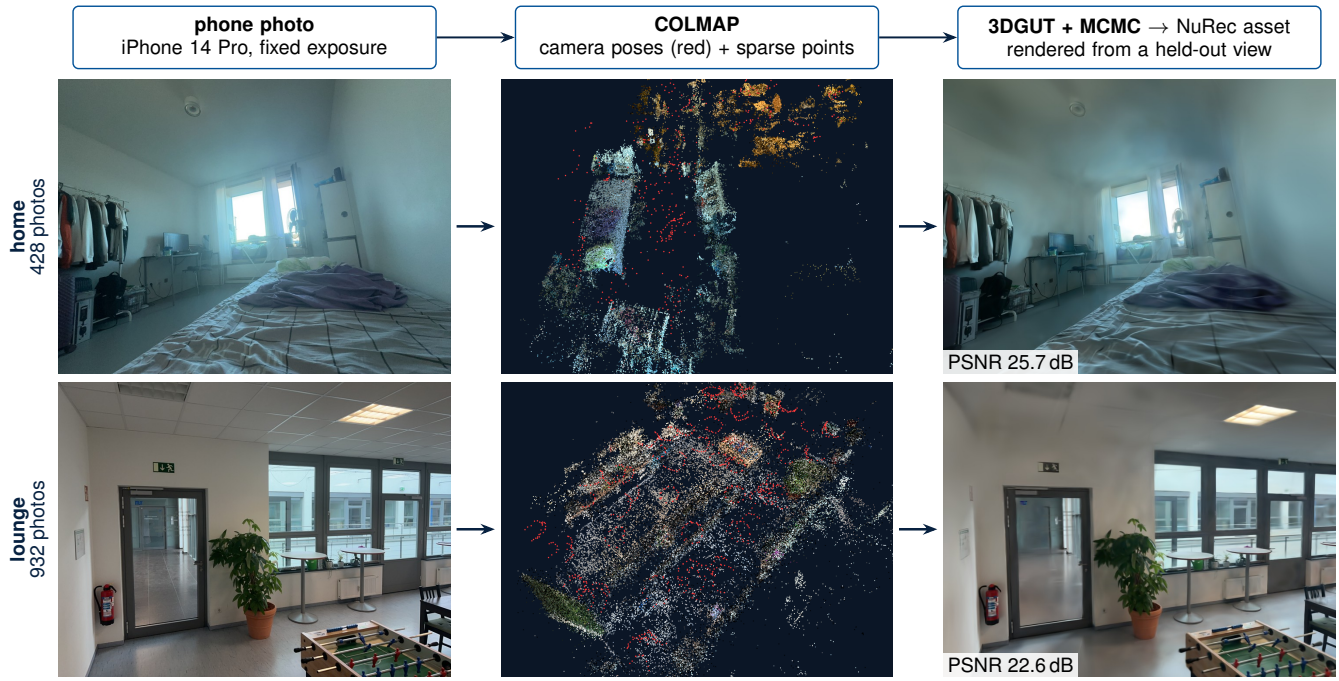


Fig. 4. From phone photos to a photoreal Isaac Sim scene, for our home (top) and a lounge (bottom). Left: one of the input photos. Middle: the COLMAP point cloud with the phone positions in red. Right: the reconstruction rendered from a view held out of training, with its PSNR. NVIDIA's own reference scene reaches 25.9 dB.



Fig. 5. Application scenarios in the Isaac Sim warehouse, rendered with our robot. (a) S1: one table with one box, and a drop zone across the aisle. (b) S2: a red, a yellow and a green table, each with a red, a yellow and a green box in random order. (c) S3: the two views DualVLN receives for *"Walk to the yellow table and stop in front of it"*: the head camera (top) and the same camera tilted 30° down (bottom), a virtual camera that gives DualVLN the downward view it was trained to use before choosing a goal, with the path it predicts in green.

side the arm needs (Fig. 1). It sees the world through the robot's own lidar, filtered so that the robot's raised arm and a carried box do not count as obstacles. A small relay raises very slow sideways and turning commands to the minimum the walking policy reacts to. Nav2 also drives the robot back to the drop zone. *Language:* DualVLN [20] receives a navigation sentence such as *"Walk to the green table and stop in front of it"* together with the front camera images, and walks towards the table; Nav2's path follower drives along its short path predictions. When DualVLN stops, the robot backs away from the table and Nav2 parks it as above. We matched DualVLN's training setup: the same 79° camera (a centre crop of the D455 image), complete sentences, a paused simulation while the model thinks, and a view looking down. DualVLN was trained in the Habitat simulator, where the agent tilts its camera 30° down before it chooses a goal point and marks that point in the downward image. Our robot cannot tilt its head, so the simulation adds a second, virtual camera at the same position, pitched 30° down, and sends both views with every request; DualVLN uses the downward one when it asks for it (Fig. 5c).

3) Manipulation: at the pick point the robot stands parallel to the table, crouches by 6 cm and unfolds the arm. Damped least-squares inverse kinematics (IK) [37] then moves the gripper to the box from the side; the gripper closes slowly and the arm lifts the box (Fig. 6). The box pose comes from the simulator. We pick from the side because from the front the robot's hips touched the table (Section III-E).

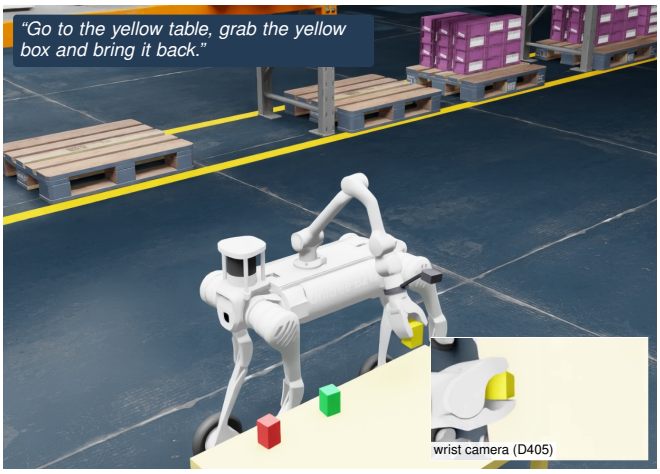


Fig. 6. The side pick in the three-table scene. The robot stands parallel to the table, crouches and grasps the requested box from the side. The inset shows what the wrist camera on our printed holder records at the same moment.

F. Recording Data for VLA Training

Every episode can be recorded at 10 Hz and converted to the LeRobot v2.1 format [25], which VLA training code such as openpi's π_0 and $\pi_{0.5}$ [22], [23] reads directly. Following [1], each frame stores

1) **vision:** RGB and depth images of the front D455 and the wrist D405;
2) **language:** the task sentence and DualVLN's navigation sentence;
3) **proprioception:** positions and velocities of all 23 joints, the base velocity, the gravity direction, the base height and the hand pose (63 values);
4) **action:** the whole-body command, i.e. base velocity, hand target pose and gripper, plus arm joint targets as an alternative;
5) **labels:** the phase (walk, Nav2, dock, pick, deliver), the planned path ahead, the target and success.

Histories and action chunks are cut from the neighbouring frames when the data is loaded, so they need no extra storage.

III. Results and Discussion

A. Experimental Setup

All experiments run on one lab workstation (Table II). The simulation parts run in their own Docker containers: Isaac Sim with the ROS 2 bridge, Isaac Lab with the robot_lab extension [38] for training, Nav2 and a server for the DualVLN model. The containers keep CUDA and driver versions apart and make it easy to move the setup to another machine or to the lab's Slurm cluster. The first problem was a silent one: ROS 2 messages from the container never reached the host, because Docker blocks the shared-memory transport of FastDDS. A UDP-only profile fixed it. A small differential-drive robot with an RGB-D camera and a 3D lidar served as a first test of sensor topics, transforms and teleoperation in RViz2.

TABLE II
Workstation and software used for all experiments.

Hardware	
CPU, RAM	AMD Ryzen 9 9950X (16 cores), 128 GB
GPU	NVIDIA RTX 6000 Ada, 48 GB (driver 570, CUDA 12.8)
OS	Ubuntu 22.04, Docker containers per component
Software	
Isaac Sim 5.1.0	scenarios, sensors, ROS 2 bridge
Isaac Sim 6.0.1	photoreal scenes (NuRec)
Isaac Lab 2.3.2	RL training with robot_lab 2.3.2 and PyTorch 2.7
ROS 2 Jazzy	Nav2, RViz2, bridges to the simulator
DualVLN	InternVLA-N1 model served next to the simulator, about 21 GB of GPU memory
COLMAP 4.2	camera poses and sparse points
3DGRUT 2.0	Gaussian training (3DGUT + MCMC), NuRec export
LeRobot	dataset format v2.1, checked with lerobot 0.3.3

Policies are trained with 4096 robots in parallel on the GPU, with physics at 200 Hz and the policy at 50 Hz; about 4000 training iterations take two hours. All fetch results come from seeded simulation runs with CPU physics, one robot at a time. An episode counts as a success when the robot picks the requested box, lifts it and still holds it at the end. We also count deliveries and falls. Reconstructions are scored with the peak signal-to-noise ratio (PSNR) on photos held out from training.

B. Photoreal Scenes

On views held out from training, the home reaches a PSNR of 25.7 dB, about the same as NVIDIA's own reference scene (Fig. 4). The lounge reaches 22.6 dB, mostly because of large blank walls. The biggest gains came from the way the photos were taken, not from more compute: more photos, the phone's ultra-wide lens, JPEG instead of RAW, and fixed exposure, focus and white balance. More Gaussians did not help: the best models had only 1.5 to 3 times as many Gaussians as COLMAP points, and much larger ones filled the room with fog. The assets carry appearance only: they have no collision geometry and no metric scale yet, so the robot cannot interact with them.

C. Whole-Body Control

A single policy that drives and reaches at the same time worked better than two policies side by side: the hand stayed steadier while driving, and the robot followed turn commands much more closely. With the hand target fixed in the world, the policy learned to use its body: it crouches by up to 30 cm and leans forward to reach low targets (Fig. 7). This is necessary because the Z1 alone cannot reach more than about 9 cm below its shoulder. The final 6D policy reached 10 of 12 test targets at 1–3 m without a fall. Watching the robot showed problems the numbers missed, such as squatting, sitting back or driving backwards, and a shaking hand. The shaking came from the policy switching the arm hard between two targets, and it disappeared once the policy commanded small changes of the arm targets instead of the targets themselves.

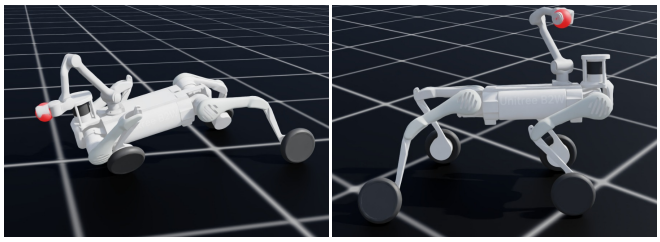


Fig. 7. The final 6D whole-body policy (robot without the gripper) reaching hand targets fixed in the world (red). Left: for a target 0.42 m above the floor, the robot crouches by about 30 cm and leans forward. Right: for a target almost 3 m behind it, it turns round, drives there and reaches up. The arm stays clear of the lidar mast.

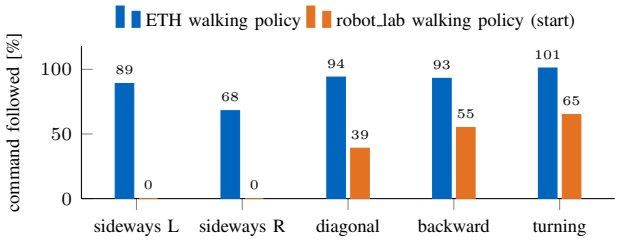


Fig. 8. How much of a velocity command the walking policy actually follows, with the arm attached (15 tests per policy). Small sideways and diagonal commands are exactly what parking at a table needs.

D. Walking Controller

ETH's walking policy follows sideways, diagonal, backward and turning commands far better than the robot_lab walking policy we started with (Fig. 8). Our fine-tune fixed the narrow stance after turns, and driving and turning stayed as good as before. Only small sideways commands from standstill are followed less well, which the command relay of the Nav2 pipeline makes up for.

E. Fetch Across Scenarios

Table III summarises the fetch tests. Turning the front pick into a side pick removed all contacts between the robot's hips and the table, and switching to the ETH walking policy with a crouch during the pick raised the success rate to 96 %. The three remaining failures were the arm not reaching the grasp pose. Nav2 brought the robot to the pick point in all 24 test runs, as reliably as the hand-tuned script. It needed no scene-specific waypoints, and the same setup can run on the real robot with its real lidar. In the three-table scene both pipelines always went to the named table and picked the named box without touching the neighbouring ones.

In the final language pipeline the robot picked the requested box in all 30 recorded episodes and delivered it in all 15 episodes that asked for delivery. DualVLN on its own is the weak link: it headed for the named table in 18 of the 30 episodes and stopped within 1 m of it in 12. Nav2 recovered every time. Part of the difficulty is the scene: from many starts the target table is out of view, and under the warehouse lights the yellow and green tables look pale. Matching DualVLN's training setup (Section II-E) mattered: with the head camera unchanged, it drove past the table.

TABLE III

FETCH RESULTS ACROSS SCENARIOS, NAVIGATION PIPELINES AND WALKING CONTROLLERS. THE ROBOT DID NOT FALL IN ANY OF THESE EPISODES.

Scen.	Navigation and pick	Walking	Ep.	Picked	Delivered
S1	scripted, front pick	RL	72	65	65
S1	scripted, side pick	RL	72	68	68
S1	scripted, side pick + crouch	ETH	72	**69**	**69**
S1	Nav2 parking	ETH-FT	24	**24**	–
S2	scripted, random target	ETH-FT	12	**12**	–
S2	Nav2 parking + delivery	ETH-FT	12	**12**	**12**
S1	scripted (data recording)	ETH	220	214	214
S3	DualVLN + scripted parking	ETH	240	230	230
S3	DualVLN + Nav2 (final)	ETH-FT	30	**30**	**15/15**

RL: robot_lab walking policy. ETH-FT: ETH's policy after our stance fine-tune. Final: half of the 30 tasks ask for delivery.

TABLE IV

FOLLOWING DUALVLN'S PREDICTED PATHS (SIX STARTS EACH, NAV2 PARKS AFTERWARDS).

	Pure pursuit	FollowPath	NavigateToPose
Reached the table (<1 m)	4/6	**6/6**	1/6
Walking time	**16 s**	29 s	34 s
Picked afterwards	6/6	6/6	6/6

F. Following DualVLN's Path

DualVLN predicts a short path about every 0.4 s, and something has to follow it. Table IV compares our own pure-pursuit follower with two Nav2 options on the same six starts. Nav2's FollowPath brought DualVLN to the table every time but drives more slowly, because it slows down at the end of each short path. NavigateToPose re-plans to a new goal every 0.4 s and drives twice the distance. We use FollowPath in the final pipeline.

G. Recorded Datasets

All datasets are stored in the LeRobot v2.1 format at 10 Hz and contain successful episodes only. Fig. 9 shows what the recorder stores for the episode of Fig. 1. The first set, made with an older recorder without depth, holds 444 episodes (7.5 h) of S1. It mixes scripted expert episodes with episodes in which DualVLN walks, which matches the two stages of [1]. The final set holds 30 episodes (41 min) of the three-table scene with RGB-D images from the D455 and the D405 and the pick-only and delivery mix, and it loads with the official LeRobot data loader. Recording takes about four minutes per episode with one simulation, so larger datasets need several simulations in parallel, each with its own Nav2 instance.

H. Discussion

Lessons. Most of the time went into faults that produced a wrong result without any error message: a ROS 2 transport that silently failed, an invisible gripper, legs that snapped straight and launched the robot at every reset because joints without a command are driven to zero, a COLMAP run whose first model held only two photos, and two simulators

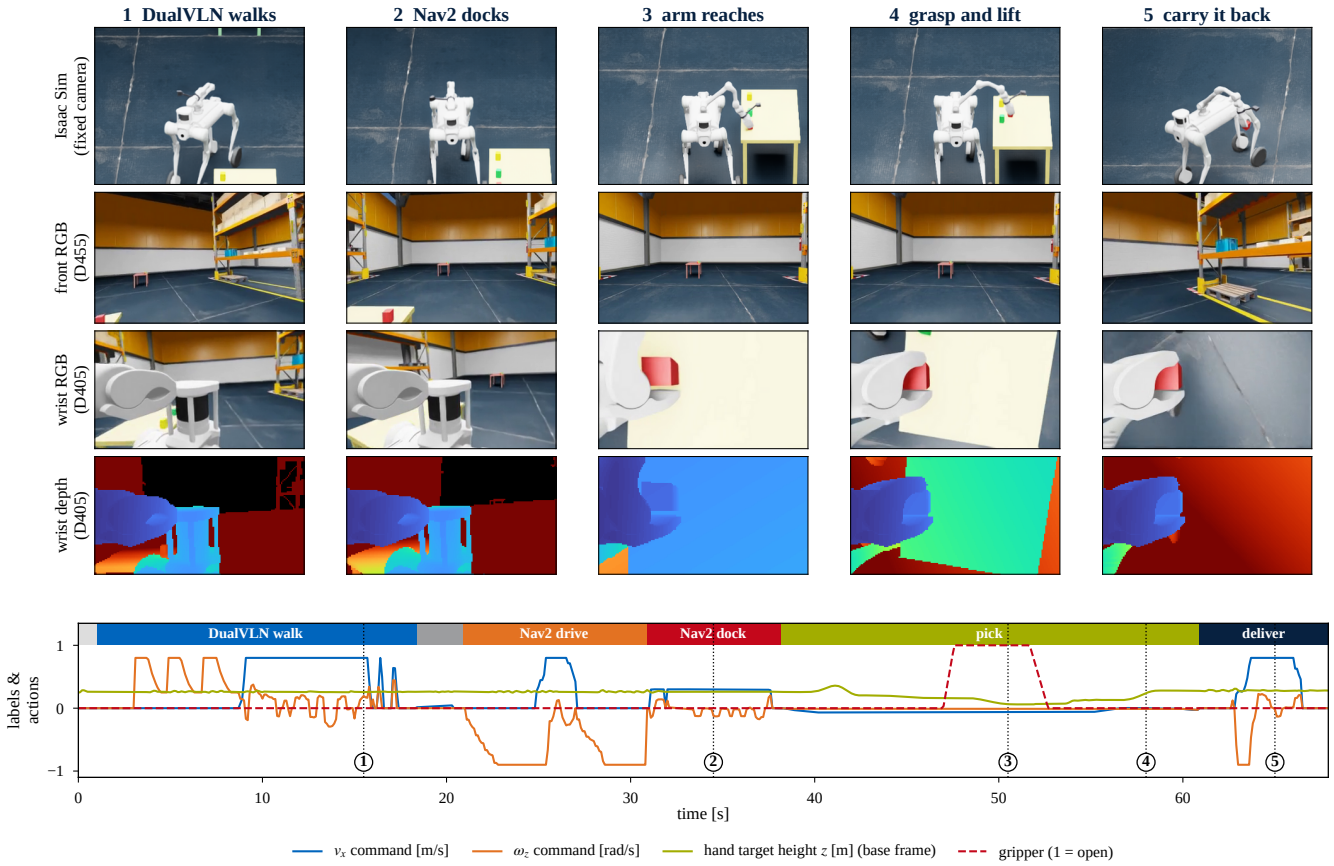


Fig. 9. The episode of Fig. 1 as the robot recorded it, *"Pick up the red box on the yellow table and bring it to me"*. Top: the robot in Isaac Sim at five moments, cut from the fixed camera of Fig. 1. Below each moment, what the robot recorded at that instant: the front RGB image of the D455, the RGB and depth images of the wrist D405 (black: no depth, e.g. closer than 7 cm). Bottom: recorded phase labels and actions over time; the numbers mark the five moments.

that confused one Nav2 instance. Three habits helped the most: watching the robot instead of only the numbers, changing one thing at a time, and testing the test itself.

Limitations. The box pose comes from the simulator, the pick is scripted, and the whole-body policies of Section II-C are not yet used in the fetch pipeline. From far starts DualVLN finds the named table in only about 60 % of the episodes. The reconstructed rooms cannot be touched by the robot, the warehouse floor is flat, and lighting and object appearance are not randomised. DualVLN's look-down view comes from a virtual camera: the real B2W cannot tilt its head, so it will need a second camera pitched down, or the body pitched forward while DualVLN looks. The final dataset is small, and nothing has run on the real robot yet.

IV. Conclusion and Future Work

We built a simulation of a wheeled quadruped manipulator that goes from a sentence to a delivered box. The robot finds the table with a language-guided navigation model, parks with Nav2, picks the box with its arm and brings it back, and every step is recorded as training data for a whole-body VLA model. In more than 800 simulated episodes the robot never fell, and in the final 30 recorded episodes it picked the requested box every time. The robot model, its sensors, two photoreal rooms and the whole-body controllers are ready to be reused for other tasks.

Future work, in order of value: record a few hundred to a thousand episodes with the final setup, using several simulations in parallel; fine-tune $\pi_{0.5}$ on the Slurm cluster with the whole-body action and test it in closed loop in the same scenes; replace the simulator's box pose with a detector on the head and wrist cameras; use the 6D whole-body policy for the pick; build collidable, metric reconstructions of real rooms; and move to the real B2W, which will carry the same cameras, lidar and printed holder.

Acknowledgment

I thank Yuan Gao and Prof. Dr.-Ing. Johannes Betz for their supervision and the TUM AVS lab for its compute.

References

[1] Y. Gao, M. Piccinini, and J. Betz, "Towards whole-body VLA: A scalable data collection framework for quadrupedal mobile manipulators," in *Proc. 2nd German Robotics Conf.*, Cologne, Germany, 2026, pp. 455–457.

[2] NVIDIA, "Isaac Sim 5.1," https://developer.nvidia.com/isaac/sim, 2025.

[3] M. Mittal, P. Roth, J. Tigue, A. Richard, O. Zhang, P. Du, A. Serrano-Muñoz, X. Yao, R. Zurbrügg, N. Rudin *et al.*, "Isaac Lab: A GPU-accelerated simulation framework for multi-modal robot learning," *arXiv preprint arXiv:2511.04831*, 2025.

[4] V. Makoviychuk, L. Wawrzyniak, Y. Guo, M. Lu, K. Storey, M. Macklin, D. Hoeller, N. Rudin, A. Allshire, A. Handa, and G. State, "Isaac Gym: High performance GPU-based physics simulation for robot learning," *arXiv preprint arXiv:2108.10470*, 2021.

[5] N. Rudin, D. Hoeller, P. Reist, and M. Hutter, "Learning to walk in minutes using massively parallel deep reinforcement learning," in *Proc. Conf. Robot Learn. (CoRL)*, 2022, pp. 91–100.

[6] A. Szot, A. Clegg, E. Undersander, E. Wijmans, Y. Zhao, J. Turner *et al.*, "Habitat 2.0: Training home assistants to rearrange their habitat," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2021.

[7] C. Li, R. Zhang, J. Wong, C. Gokmen, S. Srivastava, R. Martín-Martín *et al.*, "BEHAVIOR-1K: A benchmark for embodied AI with 1,000 everyday activities and realistic simulation," in *Proc. Conf. Robot Learn. (CoRL)*, 2022.

[8] S. Tao, F. Xiang, A. Shukla, Y. Qin, X. Hinrichsen, X. Yuan *et al.*, "ManiSkill3: GPU parallelized robotics simulation and rendering for generalizable embodied AI," *arXiv preprint arXiv:2410.00425*, 2024.

[9] C. D. Bellicoso, K. Krämer, M. Stäuble, D. Sako, F. Jenelten, M. Bjelonic, and M. Hutter, "ALMA – articulated locomotion and manipulation for a torque-controllable robot," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2019, pp. 8477–8483.

[10] J.-P. Sleiman, F. Farshidian, M. V. Minniti, and M. Hutter, "A unified MPC framework for whole-body dynamic locomotion and manipulation," *IEEE Robot. Autom. Lett.*, vol. 6, no. 3, pp. 4688–4695, 2021.

[11] L. Molnar, J. Cheng, G. Fadini, D. Kang, F. Zargarbashi, and S. Coros, "Whole-body inverse dynamics MPC for legged loco-manipulation," *IEEE Robot. Autom. Lett.*, vol. 11, no. 1, pp. 898–905, 2026.

[12] Z. Fu, X. Cheng, and D. Pathak, "Deep whole-body control: Learning a unified policy for manipulation and locomotion," in *Proc. Conf. Robot Learn. (CoRL)*, 2023, pp. 138–149.

[13] M. Liu, Z. Chen, X. Cheng, Y. Ji, R. Yang, and X. Wang, "Visual whole-body control for legged loco-manipulation," *arXiv preprint arXiv:2403.16967*, 2024.

[14] Y. Ma, F. Farshidian, T. Miki, J. Lee, and M. Hutter, "Combining learning-based locomotion policy with model-based manipulation for legged mobile manipulators," *IEEE Robot. Autom. Lett.*, vol. 7, no. 2, pp. 2377–2384, 2022.

[15] K. Jiang, Z. Fu, J. Guo, W. Zhang, and H. Chen, "Learning whole-body loco-manipulation for omni-directional task space pose tracking with a wheeled-quadrupedal-manipulator," *IEEE Robot. Autom. Lett.*, 2024, arXiv:2412.03012.

[16] M. Bjelonic, R. Grandia, O. Harley, C. Galliard, S. Zimmermann, and M. Hutter, "Whole-body MPC and online gait sequence generation for wheeled-legged robots," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, 2021, pp. 8388–8395.

[17] J. Lee, M. Bjelonic, A. Reske, L. Wellhausen, T. Miki, and M. Hutter, "Learning robust autonomous navigation and locomotion for wheeled-legged robots," *Sci. Robot.*, vol. 9, no. 89, p. eadi9641, 2024.

[18] P. Anderson, Q. Wu, D. Teney, J. Bruce, M. Johnson, N. Sünderhauf, I. Reid, S. Gould, and A. van den Hengel, "Vision-and-language navigation: Interpreting visually-grounded navigation instructions in real environments," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2018, pp. 3674–3683.

[19] J. Krantz, E. Wijmans, A. Majumdar, D. Batra, and S. Lee, "Beyond the nav-graph: Vision-and-language navigation in continuous environments," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2020.

[20] M. Wei, C. Wan, J. Peng, X. Yu, Y. Yang, D. Feng, W. Cai, C. Zhu, T. Wang, J. Pang, and X. Liu, "Ground slow, move fast: A dual-system foundation model for generalizable vision-and-language navigation," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2026.

[21] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. P. Foster, P. R. Sanketi, Q. Vuong *et al.*, "OpenVLA: An open-source vision-language-action model," in *Proc. Conf. Robot Learn. (CoRL)*, 2025, pp. 2679–2713.

[22] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai *et al.*, "π_0: A vision-language-action flow model for general robot control," *arXiv preprint arXiv:2410.24164*, 2024.

[23] Physical Intelligence, K. Black, N. Brown, J. Darpinian, K. Dhabalia, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai *et al.*, "$\pi_{0.5}$: A vision-language-action model with open-world generalization," *arXiv preprint arXiv:2504.16054*, 2025.

[24] Open X-Embodiment Collaboration, A. Padalkar *et al.*, "Open X-Embodiment: Robotic learning datasets and RT-X models," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2024.

[25] R. Cadene, S. Alibert, A. Soare, Q. Gallouedec, A. Zouitine, and T. Wolf, "LeRobot: State-of-the-art machine learning for real-world robotics in PyTorch," https://github.com/huggingface/lerobot, 2024.

[26] D. Wang, Z. Liang, H. Ye, Z. Sun, Z. Lu, Y. Zhang, Y. Zhao, Y. Gao *et al.*, "Target-Bench: Can video world models achieve mapless path planning with semantic targets?" *arXiv preprint arXiv:2511.17792*, 2025.

[27] Intel RealSense, "realsense-ros: ROS wrapper and camera descriptions for Intel RealSense cameras," https://github.com/IntelRealSense/realsense-ros, 2025.

[28] NVIDIA, "Neural reconstruction (NuRec) for robotics: Mono-camera workflow," https://docs.nvidia.com/nurec/robotics/neural_reconstruction_mono.html, 2026.

[29] J. L. Schönberger and J.-M. Frahm, "Structure-from-motion revisited," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2016, pp. 4104–4113.

[30] Q. Wu, J. Martinez Esturo, A. Mirzaei, N. Moenne-Loccoz, and Z. Gojcic, "3DGUT: Enabling distorted cameras and secondary rays in Gaussian splatting," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2025.

[31] S. Kheradmand, D. Rebain, G. Sharma, W. Sun, Y.-C. Tseng, H. Isack, A. Kar, A. Tagliasacchi, and K. M. Yi, "3D Gaussian splatting as Markov chain Monte Carlo," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2024.

[32] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis, "3D Gaussian splatting for real-time radiance field rendering," *ACM Trans. Graph.*, vol. 42, no. 4, 2023.

[33] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

[34] F. Yang *et al.*, "Spatially-enhanced recurrent memory for long-range mapless navigation via end-to-end reinforcement learning," *arXiv preprint arXiv:2506.05997*, 2025.

[35] S. Macenski, F. Martín, R. White, and J. Ginés Clavero, "The marathon 2: A navigation system," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, 2020.

[36] S. Macenski, S. Singh, F. Martín, and J. Ginés, "Regulated pure pursuit for robot path tracking," *Auton. Robots*, vol. 47, 2023.

[37] S. R. Buss, "Introduction to inverse kinematics with Jacobian transpose, pseudoinverse and damped least squares methods," Univ. of California, San Diego, Tech. Rep., 2004.

[38] Z. Fan, "robot_lab: RL extension library for robots, based on IsaacLab," https://github.com/fan-ziqi/robot_lab, 2024.